

Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 2: Function, iterations, a data structures

Aurélien Sallin (lecturer) and Federica Mascolo (class tutor)

10/10/2024

Exercises

Exercise A: Write a Sum Function

In the lecture on Programming with Data, we have discussed the most basic programming concepts in R. Among other things, you have learned how we can use for-loops to sum up numbers stored in a numeric vector.

1. Use the code example in the respective lecture notes and the following empty function construct to implement a function that takes a numeric vector as input and returns the sum of the vector's elements.

```
my_sum <- function(x){  
  
}
```

2. Define the vector `numbers = c(1, 2, 3)`. Then, compare your function `my_sum()` with the `sum()` function (the pre-implemented sum function in R) when using `numbers` as the input argument: Do you get the same output for `my_sum(numbers)` and `sum(numbers)`?

Exercise B: Robustness and Warnings

1. Once you have implemented and successfully tested `my_sum()`, call your function with the following vector as input. What happens?

```
numbers2 <- c("1", "2", "3")
```

2. Why do we get an error? Investigate by comparing the vector `numbers` from Exercise A with the new vector `numbers2`. Hint: the functions `str()` and `class()` might help you.
3. Extend your `my_sum()` function with a control statement that checks whether the input is of the right class for the function to work (Hint: `class(x)!="numeric"`). The goal is to make the function more robust to wrong input. That is, if the input is wrong, issue a warning but don't break down with an error. Use the following code for the warning message:

```
warning("Wrong input! This function only accepts numeric or integer values!")
```

4. Test your enhanced sum function with the two input vectors `numbers` and `numbers2`.

Exercise C: use the sum function and lapply/loop/map on a list of vectors

In this exercise, you will learn how to apply a custom function to multiple elements in a list using different methods.

1. use the `my_sum()` function created earlier

2. create a list of numeric vectors

```
# Create a list of vectors
list_of_numbers <- list(
  c(1, 2, 3),
  c(4, 5, 6),
  c(7, 8, 9, 10)
)
```

3. Use the `lapply()` function to apply `my_sum()` to each vector in the `list_of_numbers` list. Assign the result to a new variable called `sums_lapply`. Print the result.
4. Use a for loop to achieve the same result as in Step 3. Initialize an empty vector called `sums_loop` and then iterate over each vector in `list_of_numbers`, storing the sums in `sums_loop`. Print the result.
5. Use `purrr::map()` to Apply `my_sum()` to Each Vector Use the `map()` function from the `purrr` package to apply `my_sum()` to each vector in `list_of_numbers`. Assign the result to a variable called `sums_map`. Make sure to load the `purrr` package before using `map()`. Print the result.

Exercise D: Data structures

Vectors, factors and lists

1. Create a vector `student_age` containing the ages of 5 students.
 - print its length and type using `length()` and `typeof()` functions
2. Create a factor `student_grades` representing the grades of the same 5 students: “A”, “B”, “A”, “C”, “B”.
 - print the factor levels, why do you use factors?
3. Create a list `student_info` that combines `student_ages`, `student_grades`, and another vector `student_names` containing the names of the students.
 - access individual elements using both numeric and named indexing.
4. What is the difference between a vector and a factor? Why do you use a list instead of a vector when dealing with different types of data?

Matrices and arrays

1. Create a matrix `scores_matrix` representing the scores of 3 students in 3 subjects
 - give row and column names using `rownames()` and `colnames()`.
 - print the matrix and the score of a student that you prefer.
2. Create an array `student_array` to store the same data as in `scores_matrix`, but this time, add scores from another semester. Use the same students and subjects but with different scores.
 - this 3-dimensional array, access “Student 3” scores in the first subject listed across both semesters.
3. What are the differences between a matrix and an array? When would you use an array instead of a matrix?

Data Frames, tibble and data.table

1. Create a data frame `students_df` combining all the data for 5 students: `student_names`, `student_ages`, `student_grades`, and another vector `attendance` containing their class attendance (in percentage): 85, 92, 78, 88, 90.
 - access the columns of attendance.
 - access the data for student 3.
2. Why is a data frame more suitable for this kind of structured data than a matrix?
3. Convert the existing `students_df` data frame into a tibble using the `as_tibble()` function.
 - name the tibble `students_tibble` and print the data frame and the tibble.

4. Create a `data.table` named `students_dt` from `students_df` using the `as.data.table()` function from the `data.table` package.
 - convert `students_df` to a `data.table`
 - select a column (`ages`), filter rows (`grades` equal to “A”)

Additional Exercises for Training

5. Use the output from the previous exercise. Compare the syntax of common data manipulation tasks (e.g., filtering, grouping, summarizing) between `data.frame`, `tibble`, and `data.table`.
 - Filter students with attendance greater than 85
 - Calculate mean age of students who passed

Exercise E: Standard Deviation Function

1. During the lecture on Programming with Data we have step-by-step implemented our own R function to compute the mean. Now implement an R function called `my_sd` that computes the standard deviation, defined as: $SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$. Hint: revisit the math operators section in the notes on the first exercise session.
2. Test your implementation by comparing its output with the already implemented `sd()` function.

Exercise F: Standard Error Function

Building on the function written in Exercise D, implement a function to compute (the estimate of) the sample mean standard error as defined here: $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$. Hint: the idea is to use your own implementation (in Exercise D) to compute SD as a first step of your standard error function.

Exercises D and E should illustrate how you can approach more complex problems by breaking them up in smaller problems and addressing each of these step-by-step. That is, when looking at the problem of computing $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$, you see that computing SD is a part of the overall problem, which can be addressed separately.

Exercise G: T-test

Implement a function for the one-sample t-test: $t = \frac{\bar{x} - \mu_0}{SE}$.